



Using AI for Source Code Vulnerability Analysis

A workflow and kit for security teams directing language models to discover and exploit vulnerabilities through source code assessment.

The five-step loop

1

MAP:

Have the AI inventory entry points, auth boundaries, and data flows.

2

SLICE:

Have the AI pull a dependency-complete fragment with its middleware, decorators, and config.

3

INTERROGATE:

Ask the AI adversarial questions, not descriptive ones.

4

VERIFY:

Reproduce each AI-claimed finding by hand. This step stays human-owned.

5

EXPLOIT:

Work with the AI to chain validated findings into working attacks with captured impact.

TIP: Skipping steps produces findings that will not survive exploitation or report review.

Prompt patterns

Useful prompt framings when asking the AI about a code slice:

- **Threat-model a slice:** Ask the AI to list each place attacker-controlled input reaches a sensitive destination.
- **Authorization tracing:** Have the AI trace the checks that run before a handler executes (and where they could be skipped).
- **Differential reading:** Feed the AI two similar handlers and request a side-by-side comparison of validation and authorization.

- **Negative-space search:** Ask the AI to surface state-changing operations that do not check caller identity.
- **Constraint prompting:** Bound the AI's scope. Tell it what it may assume, what to ignore, and when to stop.
- **Tool generation:** Ask the AI to write a small custom scanner or parser for the specific question at hand.

TIP: Adversarial framing produces findings. Descriptive framing produces rewrites of the code.

Hallucination defenses

Treat every AI-produced finding as a hypothesis until reproduced.

- **Require citations:** Each claim needs a file:line reference, or discard it.
- **Reset on fabrication:** Invented functions or vulnerability identifiers contaminate the session. Start over.
- **Crosscheck important findings:** Run the slice past a second model from a different vendor.
- **Allow "I don't know:":** Instruct the model to say so rather than guess.

TIP: Confident, unreproducible findings damage the assessment and the reviewer's credibility.

Before uploading source

Validate these checks before any code leaves the reviewer's machine:

- **Confirm project scope:** Analysts need written permission to send client source to a third-party model.
- **Check internal policy:** Internal reviewers should verify company data classification and acceptable use rules for generative AI.
- **Know the provider's retention:** Understand whether prompts are logged, retained, or used for training.
- **Default to private:** When any of the above is unclear, use a local model or a zero-retention enterprise tenant.

TIP: Cloud-hosted models process whatever reviewers send them. Assume anything uploaded is visible to the provider.